

Weekly Diary – Week 16

Internship Organization: Vikaspedia

Duration: 22/04/2026 – 26/04/2026

Objective of the Week

The objective of this week was to study the requirements of the Broken Link Detection & Auto Replacement System and begin development of the core crawling and link validation modules. The focus was on understanding large-scale URL processing, HTTP request handling, and scalable crawling strategies for detecting broken and redirected links efficiently.

Date- 22/April

Studied the complete project requirement document for the Broken Link Detection & Auto Replacement System. Analyzed the major objectives of the project such as scanning more than 50,000 URLs, detecting broken links, handling redirects, and suggesting replacement links automatically. Understood the importance of scalability, performance optimization, and concurrency handling in large-scale crawling systems. Also researched how modern crawlers validate URLs using HTTP requests and how status codes such as 200, 301, 404, and 500 are used to determine whether a link is valid or broken. Prepared notes regarding the overall workflow and system architecture.

Date- 23/April

Researched the technical concepts required for building a large-scale link crawler. Studied how HTTP HEAD and GET requests are used for validating URLs and compared their performance differences. Explored timeout handling, DNS failures, redirect chains, and retry mechanisms that are commonly used in web crawling systems. Also analyzed how excessive requests can overload servers and why rate limiting and batch processing are necessary for maintaining stable crawler performance. Prepared the initial development environment and project structure for implementing the crawler system.

Day- 24/April

Designed the architecture of the link crawling engine and planned the processing flow for URL validation. Implemented the initial backend structure required for processing URLs and recording validation results. Studied queue-based processing techniques for managing large numbers of URLs efficiently. Also explored how multi-threading can improve performance by processing multiple links simultaneously instead of sequentially. Conducted small experiments with sample URLs to understand how HTTP responses are captured and interpreted programmatically.

Date- 25/April

Implemented the first version of the link validation module using HTTP request handling techniques. Developed functionality to send HEAD requests for checking URL availability and fallback to GET requests whenever required. Added logic for capturing response codes, redirects, timeout failures, and invalid domains. Conducted testing with sample URLs to verify whether the system could correctly classify valid, broken, and redirected links. Also improved exception handling to ensure the crawler continues processing even when some requests fail unexpectedly.

Date- 26/April

Performed testing of the initial crawler implementation using different categories of sample URLs. Added redirect-following functionality so the crawler can validate the final destination of redirected links. Implemented basic logging functionality for recording failed requests and response details. Studied how infinite redirect loops can affect crawler stability and implemented preliminary loop detection logic. Documented observations related to crawler performance, request speed, and response handling during testing.

Conclusion

By the end of this week, the foundational architecture of the Broken Link Detection & Auto Replacement System was successfully prepared. The initial crawler and validation modules were implemented, and the system was able to process URLs, capture HTTP responses, identify broken links, and handle redirects efficiently.